



# **Desarrollo Web Profesional**

**José Miguel Carrera Pacheco**



**Alejandro Contreras Martinez**

**Emmanuel Castro Salvador**

**Carlos Eduardo Hernandez Camacho**

**Danna Paola Tobón Mosqueda**

**Manelic Alitzel Reyes Torres**

**8.-A**

**DOM, Componentes Dinámicos y  
Accesibilidad**

**07/02/2026**

## DOM vs. Virtual DOM: Optimización y Reconciliación

El DOM (Document Object Model) es la representación estructural de un documento HTML en el navegador. Según la W3C, el DOM permite que los scripts actualicen dinámicamente el contenido, la estructura y el estilo de los documentos.

- El Problema del DOM Real: Las actualizaciones en el DOM real son costosas. Cada vez que se inserta un nodo (como una nueva fila en nuestra tabla de incidencias), el navegador debe realizar un proceso de Reflow (recalculado de geometría) y Repaint (dibujado visual), lo que impacta negativamente en el rendimiento si se hace de forma masiva.
- La Solución del Virtual DOM: Popularizado por bibliotecas como React, el Virtual DOM es una copia abstracta del DOM real. El proceso de Reconciliación utiliza un algoritmo de Diffing para comparar el estado anterior con el nuevo.
  - Impacto DevOps: Desde nuestra perspectiva, el Virtual DOM facilita la creación de Tests Unitarios de Interfaz, ya que permite renderizar componentes en entornos de prueba (como JSDOM) sin necesidad de un navegador real completo, acelerando el pipeline de CI/CD.

## Problemas Comunes al Manipular el DOM

La manipulación inadecuada, especialmente en aplicaciones de una sola página (SPA), genera fallos que comprometen la estabilidad:

- Fugas de Memoria (Memory Leaks): Ocurren cuando componentes dinámicos se desmontan pero dejan activos sus Event Listeners. Esto satura el "Garbage Collector" del navegador.
- Inconsistencia de Identificadores (IDs duplicados): Al generar componentes dinámicamente (ej. varios modales), es común repetir IDs, lo que rompe la lógica de los lectores de pantalla y scripts de automatización.
- Flaky Tests en CI/CD: Si el DOM dinámico tarda en renderizar, los tests automatizados pueden fallar aleatoriamente. Es responsabilidad de DevOps configurar tiempos de espera asíncronos (Async Utilities) para asegurar la consistencia.

## Gestión de Foco (Focus Management)

La gestión de foco es la técnica de controlar qué elemento recibe los eventos del teclado en un momento dado. En el SGIM, esto es crítico cuando aparecen alertas o modales dinámicos.

- Focus Trapping: Cuando se abre un modal de "Reporte Exitoso", el foco debe quedar atrapado dentro de él. Si el usuario presiona Tab y el foco se va a un elemento oculto detrás del modal, se rompe la accesibilidad.
- Restauración del Foco: Al cerrar un elemento dinámico, el foco debe volver al botón que lo activó originalmente.
- Validación DevOps: Implementamos pruebas que verifican que el `document.activeElement` coincida con el componente esperado tras una interacción dinámica.

## Uso Correcto de ARIA (Accessible Rich Internet Applications)

ARIA es una especificación de la WAI (Web Accessibility Initiative) que proporciona atributos adicionales para describir estados dinámicos que el HTML nativo no puede expresar.

- Roles Dinámicos: Atributos como `role="status"` o `role="alert"` notifican cambios importantes sin mover el foco.
- Propiedades de Estado: `aria-busy="true"` informa al usuario que una sección del DOM se está cargando (ej. al consultar la base de datos de PostgreSQL).
- `aria-live`: Define la prioridad con la que el lector de pantalla debe anunciar un cambio dinámico (polite vs assertive).
- Regla de Oro de ARIA: "No usar ARIA si existe un elemento HTML nativo que cumpla la función". El uso excesivo de ARIA puede confundir más de lo que ayuda si la estructura base es sólida.

## Referencias Bibliográficas:

React. (2024). Preserving and Resetting State. React Documentation. Recuperado el 7 de febrero de 2026, de <https://react.dev/learn/preserving-and-resetting-state>

Mozilla Developer Network [MDN]. (2023, 15 de diciembre). Performance: Avoiding layout thrashing. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/Performance/Tips>

WebAIM. (2022, 30 de noviembre). Keyboard Accessibility. Web Accessibility in Mind. <https://webaim.org/techniques/keyboard/>

World Wide Web Consortium [W3C]. (2023). WAI-ARIA Overview. Web Accessibility Initiative (WAI). <https://www.w3.org/WAI/standards-guidelines/aria/>